Basic Matrix Multiplication in Python, Java and C

Assignment 1 – Big Data

Pablo Martín Perdomo
DNI: 78823396G
Group: 17, 42

October 16, 2026

Abstract

This report compares the straightforward triple-loop dense matrix multiplication implemented in Python, Java and C for $n \times n$ integer matrices with n from 50 up to 1000. Each configuration was measured with five repetitions in fresh processes, recording median and variability of the runtime, peak resident memory, and correctness against a reference. All three programs reproduce the reference results exactly. In the tested environment, runtimes follow the cubic model approximately (fitted exponents between 2.89 and 3.45), CPython is roughly 90 times slower than C at $n = 750$ and exceeds the 60 s budget at $n = 1000$, while Java is between 1.5 and 1.8 times slower than C at the largest sizes. Conclusions are limited to the tested implementations, settings, inputs and hardware.

Contents

1 Introduction and objective **3**

2 Experimental setup **3**
2.1 Hardware and software environment 3
2.2 Implementations 3
2.3 Compiler, JVM and interpreter settings 4
2.4 Measurement protocol 4

3 Correctness **5**

4 Results **5**
4.1 Runtime 5
4.2 Growth compared with the cubic model 7
4.3 Memory 8
4.4 Limits 8

5 Analysis **8**

6 Threats to validity **10**

7 Reproducibility **10**

8 Conclusions **10**

A Source code **10**
A.1 C (`matrix.c`) 10
A.2 Java (`main.java`) 12
A.3 Python (`matrix.py`) 13
A.4 Peak-RSS wrapper (`memrun.c`) 14

1 Introduction and objective

Dense matrix multiplication $C = A \cdot B$ for $n \times n$ matrices requires n^3 multiply–add operations with the classical algorithm:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik}\, B_{kj}.$$

The objective is to compare three implementations of exactly this algorithm (no optimised library calls) and to investigate the effect of n on runtime and memory, discussing interpreter, runtime and memory-layout effects only where the evidence supports them, and comparing the trend with the cubic model.

2 Experimental setup

2.1 Hardware and software environment

Item	Value
CPU	Intel(R) Xeon(R) Processor @ 2.80GHz (2 virtual cores, cloud VM)
RAM	7.8 GiB
Operating system	Linux-6.18.44-fc-v139-x86_64-with-glibc2.39
C compiler	gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
Java / JVM	openjdk version "21.0.12.1" 2026-08-18 (OpenJDK 64-Bit Server VM, default flags)
Python	3.13.16 (main, Oct 1 2026, 15:40:24) [GCC 13.3.0] / CPython

Table 1: Execution environment. All measurements were taken on this machine.

The machine is a shared virtual machine, so some timing noise is expected, especially for very short runs. NumPy is used only in the test and verification scripts to compute the reference result; it is never called in any timed code.

2.2 Implementations

All three versions use the same algorithm, the same loop order (i–j–k, accumulating $\sum_k A_{ik}B_{kj}$ in a local variable and storing it once into C_{ij}), the same numeric type family (integers) and the same deterministic inputs

$$A_{ij} = (7i + 3j + 1) \bmod 10 + 1, \qquad B_{ij} = (5i + 11j + 2) \bmod 10 + 1,$$

so that the three programs multiply identical matrices and can be compared exactly. The largest entry of C at $n = 1000$ is 10^5, so 32-bit arithmetic never overflows.

Language	Storage layout	Unavoidable differences
C	`int**`: array of row pointers, each row a separate `malloc` block of 32-bit `int`	Rows contiguous, the whole matrix is not; no bounds checks; native code.
Java	`int[][]`: array of references to `int[]` rows (32-bit)	Rows contiguous; array bounds checks; code starts interpreted and is JIT-compiled at run time; garbage-collected heap.
Python	list of lists; each element is a pointer to an `int` object	Bytecode interpreter; arbitrary-precision integers; small integers (here the inputs, 1–10) are shared cached objects, but each result element is a separate heap object.

Table 2: Storage layout and language-specific differences.

2.3 Compiler, JVM and interpreter settings

- **C:** `gcc -O3 matrix.c -o matrix_c` (no `-march=native`, no `-ffast-math`).
- **Java:** `javac main.java`; run as `java main <n> <warmup>` with *default JVM flags* (no `-Xmx`, `-Xint`, `-XX` options). Two configurations are reported: *Java (cold JVM)*, with 0 warm-up runs, so interpretation and JIT compilation are part of the measurement; and *Java (3 warm-up runs)*, where the same multiplication is executed three times untimed before the timed run in the same JVM.
- **Python:** CPython 3.13.16 (main, Oct 1 2026, 15:40:24) [GCC 13.3.0] / CPython, pure Python loops (no NumPy, no C extensions); no warm-up is applicable.

Settings held fixed: algorithm and loop order, input generator, integer type family, matrix sizes, repetitions, timeout, hardware, compiler flags and JVM flags. **What differs** between the tested implementations is, by design, the language implementation (native code, JIT, interpreter).

2.4 Measurement protocol

- **Sizes:** $n \in \{50, 100, 150, 200, 250, 300, 400, 500, 750, 1000\}$.
- **Repetitions:** 5 per configuration, each in a *fresh process*; repetitions are interleaved across implementations to spread system drift. Total: 195 successful runs.
- **Time:** only the multiplication is timed, with a monotonic wall clock (`clock_gettime` in C, `System.nanoTime()` in Java, `time.perf_counter()` in Python). Input generation, process start-up and JVM start-up are excluded. We report median, min, max and the coefficient of variation (sample std / mean).
- **Memory:** metric = *peak resident set size* (maximum RSS) of the whole process, obtained with the small wrapper `memrun` (`wait4()` / `ru_maxrss`, equivalent to `/usr/bin/time -f %M`). It includes the JVM and the interpreter themselves.
- **Budget:** 60 s per run. A run exceeding the budget is killed, recorded as timeout, and larger sizes of that configuration are skipped.
- **Correctness during the benchmark:** every run prints a weighted checksum $\sum_{i,j}(i+1)(j+1)C_{ij}$ that is compared with a NumPy int64 reference.

3 Correctness

The tests (`test_correctness.py`) use exact integer arithmetic, hence the numerical tolerance is **zero** (exact equality required).

1. **Hand-computed case** ($n = 2$): $A = \begin{pmatrix} 2 & 5 \\ 9 & 2 \end{pmatrix}$, $B = \begin{pmatrix} 3 & 4 \\ 8 & 9 \end{pmatrix}$, so $C = \begin{pmatrix} 46 & 53 \\ 43 & 54 \end{pmatrix}$.
2. **Full-matrix comparison** with a NumPy reference for $n \in \{1, 2, 3, 5, 10, 16, 33, 64\}$ (including $n = 1$ and non-powers of two).
3. **Checksum comparison** with the reference for $n = 100$ and $n = 250$.

Implementation	Checks passed	Total
C	11	11
Java	11	11
Python	11	11

Table 3: Correctness tests (exact integer equality with the NumPy reference).

In addition, the checksum of every one of the 195 benchmark runs matched the reference (all runs correct).

4 Results

4.1 Runtime

n	Python (CPython)	Java (cold JVM)	Java (3 warm-up runs)	C (gcc -O3)
50	0.0076	0.0018	0.00032	0.00007
100	0.0602	0.0044	0.00094	0.00063
150	0.2058	0.0105	0.0031	0.0027
200	0.5096	0.0159	0.0073	0.0056
250	0.9752	0.0326	0.0139	0.0114
300	1.940	0.0497	0.0264	0.0210
400	5.014	0.1137	0.0895	0.0523
500	9.977	0.2194	0.1713	0.1052
750	36.863	0.7192	0.6809	0.4106
1000	*timeout*	1.741	1.730	1.124

Table 4: Median multiplication time in seconds (5 repetitions per cell).

n	Python (CPython)	Java (cold JVM)	Java (3 warm-up runs)	C (gcc -O3)
50	0.0072 / 0.0110 (19.2%)	0.0018 / 0.0020 (3.2%)	0.00030 / 0.00043 (15.3%)	0.00007 / 0.00016 (41.4%)
100	0.0573 / 0.0716 (9.6%)	0.0040 / 0.0079 (32.4%)	0.00091 / 0.0046 (97.7%)	0.00061 / 0.00067 (3.2%)
150	0.2005 / 0.2234 (4.6%)	0.0093 / 0.0176 (30.3%)	0.0026 / 0.0046 (23.7%)	0.0024 / 0.0035 (16.3%)
200	0.4648 / 0.5564 (7.9%)	0.0130 / 0.0235 (25.9%)	0.0068 / 0.0127 (30.9%)	0.0052 / 0.0068 (13.4%)
250	0.9590 / 1.015 (2.3%)	0.0298 / 0.0462 (19.1%)	0.0131 / 0.0152 (5.5%)	0.0101 / 0.0119 (6.4%)
300	1.844 / 1.976 (2.9%)	0.0427 / 0.0667 (18.6%)	0.0263 / 0.0290 (4.8%)	0.0181 / 0.0231 (9.3%)
400	4.952 / 5.203 (2.0%)	0.1082 / 0.1227 (5.4%)	0.0824 / 0.0974 (6.7%)	0.0437 / 0.0728 (20.7%)
500	9.906 / 10.147 (1.2%)	0.2113 / 0.2244 (2.4%)	0.1677 / 0.1809 (2.9%)	0.0916 / 0.1450 (19.3%)
750	35.554 / 37.719 (2.3%)	0.7160 / 0.7380 (1.2%)	0.6318 / 0.7007 (3.9%)	0.3976 / 0.4300 (3.1%)
1000	*timeout*	1.721 / 1.835 (3.2%)	1.553 / 1.945 (8.7%)	1.031 / 1.158 (5.5%)

Table 5: Variability: min / max (s) and coefficient of variation (std/mean, %).

Variability is small for long runs (coefficient of variation of a few percent for Python above $n = 250$) and large for very short runs: C below $n = 300$ and Java below $n = 400$ take milliseconds or less, and the measurements show coefficients of variation that are often 10–40% (98% for the Java warm-up case at $n = 100$, caused by a single slow repetition). Medians are therefore used throughout.

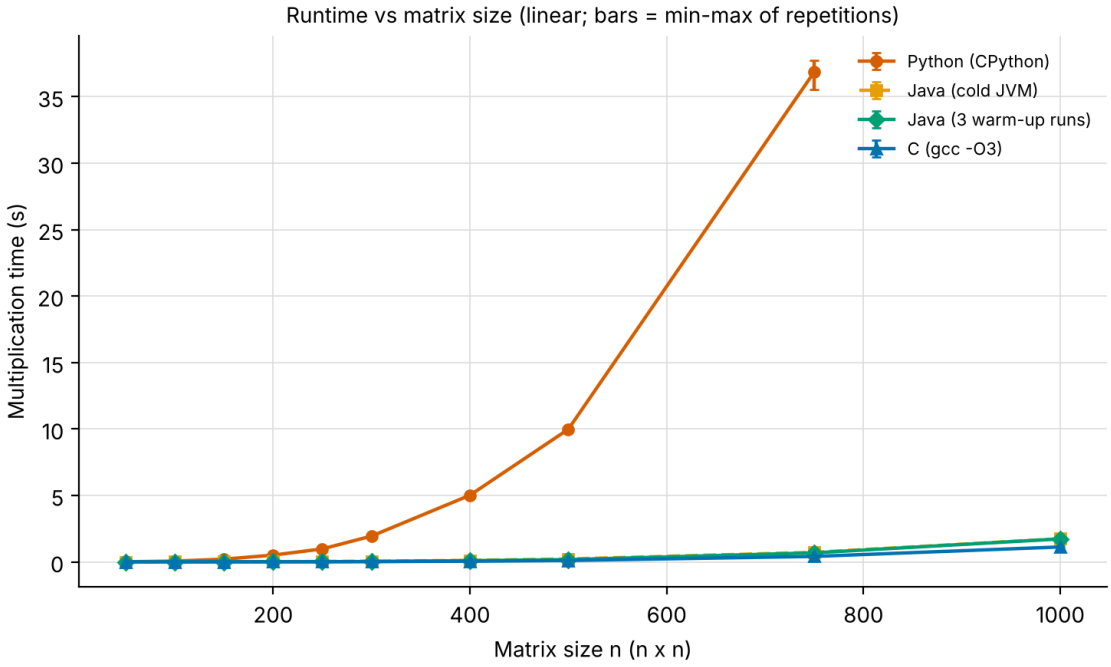


Figure 1: Runtime versus matrix size (linear axes). Python dominates; the other curves are compressed near zero.

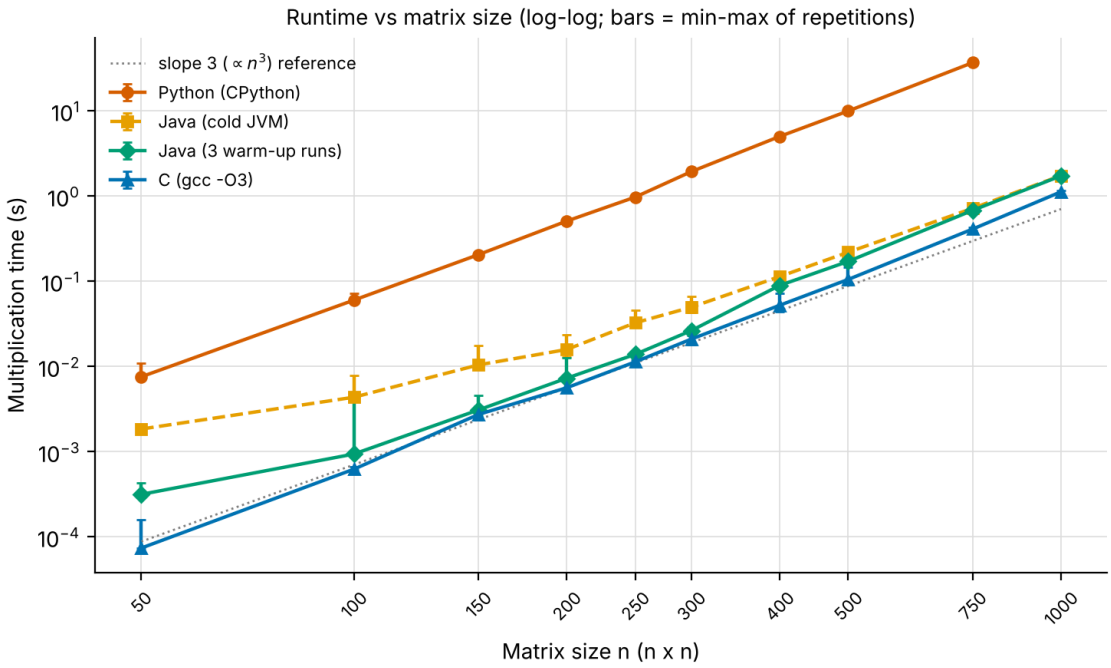


Figure 2: Runtime versus matrix size on log–log axes; error bars show min–max over 5 repetitions. The dotted line has slope 3 (cubic model) anchored at $n = 200$.

Implementation	Median at $n = 750$ (s)	Slowdown vs. C
Python (CPython)	36.863	89.8×
Java (cold JVM)	0.7192	1.8×
Java (3 warm-up runs)	0.6809	1.7×
C (gcc -O3)	0.4106	1.0×

Table 6: Relative cost at the largest size where all four configurations completed.

4.2 Growth compared with the cubic model

Implementation	Fitted exponent b	R^2	Points ($n \geq 200$)
Python (CPython)	3.26	0.9994	6
Java (cold JVM)	2.89	0.9994	7
Java (3 warm-up runs)	3.45	0.9983	7
C (gcc -O3)	3.28	0.9997	7

Table 7: Slope of $\log t$ vs. $\log n$ (least squares on medians); the cubic model predicts $b = 3$.

Implementation	$t(100)/t(50)$	$t(200)/t(100)$	$t(300)/t(150)$	$t(400)/t(200)$	$t(500)/t(250)$	$t(1000)/t(500)$
Python (CPython)	8.0	8.5	9.4	9.8	10.2	–
Java (cold JVM)	2.4	3.6	4.8	7.2	6.7	7.9
Java (3 warm-up runs)	3.0	7.8	8.5	12.2	12.3	10.1
C (gcc -O3)	8.4	9.0	7.7	9.3	9.3	10.7

Table 8: Observed growth when n doubles (ideal cubic model: 8.0).

4.3 Memory

n	Python (CPython)	Java (cold JVM)	Java (3 warm-up runs)	C (gcc -O3)
50	9.2	41.7	42.0	1.7
100	9.6	42.3	42.5	1.7
150	10.0	42.5	42.9	1.9
200	11.0	42.7	43.2	2.0
250	12.3	42.9	43.8	2.3
300	13.7	43.3	44.4	2.7
400	17.4	44.2	46.1	3.4
500	22.4	45.3	50.5	4.5
750	39.1	53.4	54.9	8.0
1000	*timeout*	56.6	63.7	13.2

Table 9: Peak resident set size (MiB, median of repetitions).

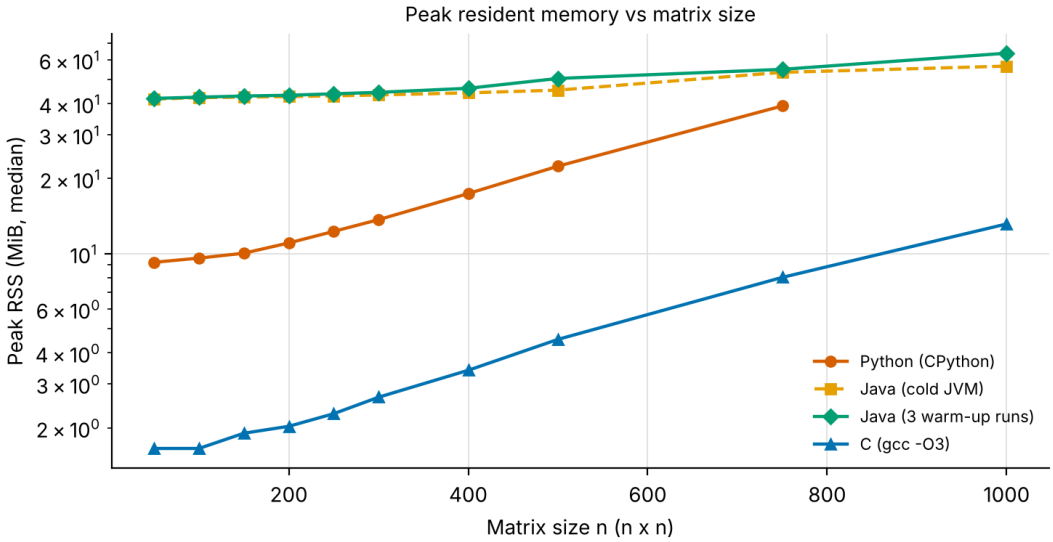


Figure 3: Peak resident set size (median of repetitions) versus matrix size.

4.4 Limits

- Largest size completed within the 60 s budget: C 1000, Java (cold) 1000, Java (warm-up) 1000, Python 750.
- Failures: Python (CPython) at $n = 1000$ (timeout). No memory failures occurred (the largest peak RSS was below 70 MiB on a machine with 7.8 GiB of RAM).
- The largest size at which all four configurations completed is $n = 750$.
- $n = 1000$ was the largest size attempted; larger sizes were not tested for C and Java.

5 Analysis

Growth versus the cubic model. The algorithm performs n^3 multiply–add operations, so doubling n should multiply the runtime by about 8 if the cost per operation is constant. Over

the sizes $n \geq 200$ the fitted log–log slopes are 3.26 (Python), 3.28 (C), 2.89 (Java, cold) and 3.45 (Java, warm-up), all with $R^2 > 0.998$. Python and C therefore grow slightly *faster* than n^3, with doubling ratios between 7.7 and 10.7 instead of 8. The trend is cubic to a first approximation, but the constant is not strictly constant. A plausible explanation is memory-hierarchy pressure: in the k loop, B is read column-wise, i.e. with a stride of one row between consecutive accesses, and the three matrices exceed the 2 MiB L2 cache of this machine once $n \approx 500$ or larger (three $n \times n$ 4-byte matrices occupy about $12\,n^2$ bytes, 3 MB at $n = 500$). This is a hypothesis consistent with the data, *not* verified: no cache counters were collected and the timer noise of a shared VM is of the same order as the effect.

Java: start-up and JIT effects. The cold-JVM slope (2.89) is below 3 only because small sizes carry a roughly fixed cost (interpretation and JIT compilation inside the timed region): at $n = 50$ the cold time is about 6× the warm-up time, and the doubling ratio rises from 2.4 to 7.9 as the fixed part becomes negligible. With three warm-up runs the Java time falls to within about 1.2–1.7× of C for $n \geq 200$; the gap between cold and warm Java disappears by $n = 1000$ (1.74 s versus 1.73 s, difference within the repetition spread), meaning that warm-up matters for runs shorter than about a second in this setup. The warm-up slope (3.45) is the highest observed and is driven by the steps between $n = 100$ and $n = 500$ (doubling ratios 7.8 to 12.3); I did not isolate its cause (candidates are JIT tier transitions and cache effects), so no explanation is claimed.

Interpreter effects. CPython executes each inner-loop iteration as several bytecodes with dynamic type checks, reference counting and allocation of a new integer object for each accumulated value, so the cost per multiply–add is much larger than in compiled code. In the tested setup the median is about 36.9 s at $n = 750$ versus 0.41 s for C (about 90×) and the ratio Python/C stays between about 85 and 100 for $n \geq 100$; the 60 s budget is exceeded at $n = 1000$ (extrapolating the $n = 750$ median with the fitted exponent gives roughly 90 s, not measured).

Memory. The measured peak RSS is dominated by the runtime for small n and by the data for large n. C starts at about 1.7 MiB and reaches 13.2 MiB at $n = 1000$; the three matrices alone need $3 \cdot 4 \cdot 10^6$ bytes = 11.4 MiB, which accounts for the growth almost exactly. The JVM has a baseline of roughly 42 MiB (heap reservation, class data and JIT) and grows to 57 MiB (cold) and 64 MiB (with warm-up); the extra memory of the warm-up configuration is consistent with the additional result matrices allocated by the untimed runs, which the garbage collector has not necessarily released. Python starts at about 9 MiB and reaches 39 MiB at $n = 750$. This agrees with a rough estimate of 8-byte pointers for A, B and C plus one 28-byte integer object per element of C (inputs are small cached integers), i.e. $(3 \cdot 8 + 28)\,n^2 \approx 29$ MB ≈ 28 MiB at $n = 750$, plus the interpreter baseline. Python's data footprint per index pair (i, j) is therefore about 4 times that of C and Java (52 versus 12 bytes).

Scope. These results apply only to the tested implementations (CPython 3.13.16 (main, Oct 1 2026, 15:40:24) [GCC 13.3.0] / CPython, OpenJDK with default flags, gcc `-O3` code), the specific input generator, the sizes $50 \leq n \leq 1000$ and this virtual machine; they do not show that one language is universally faster. Absolute times on other hardware will differ; the relative ordering and approximate growth rates are the transferable findings. Language-specific optimisations (loop interchange, blocking, `-march=native`, multithreading, `long`/`double` types, NumPy) were deliberately excluded and would have to be reported as separate experiments.

6 Threats to validity

- Shared virtual machine with 2 vCPUs: external load may inflate individual runs; the median and min–max are reported for this reason.
- Only 5 repetitions per cell; the standard deviation estimates are rough, especially for runs below 10 ms.
- Java and Python results depend on JVM and interpreter versions and defaults (JIT compiler threads run concurrently on the second core).
- The 60 s budget censors the Python series at $n = 750$; the exponent fit for Python uses 6 points.
- Peak RSS is a process-level metric and cannot separate data from runtime overhead; the decomposition above is an estimate.

7 Reproducibility

The submitted folder contains: `matrix.c`, `main.java`, `matrix.py`, `memrun.c`, `benchmark.py`, `test_correctness.py`, `analyze.py`, `report.tex`, the raw data `results.csv`, `summary.csv`, `correctness.csv`, `env.txt` and the benchmark log. Requirements: `gcc`, a JDK, Python 3 with `numpy` and `matplotlib`, and `pdflatex`.

```
pip install numpy matplotlib
python3 test_correctness.py          # 33 exact correctness checks
python3 benchmark.py --reps 5 --timeout 60    # writes results.csv, env.txt (
    about 8 minutes)
python3 analyze.py                   # figures, tables, summary.csv
pdflatex report.tex && pdflatex report.tex
```

Re-running on another machine regenerates all tables, figures and environment fields automatically; the numerical statements in the prose of Section 5 refer to the run included with this report and should be rechecked after a re-run.

8 Conclusions

In the tested environment, the three implementations produce identical, exactly verified results, and all grow approximately cubically in n (fitted exponents 2.9–3.5; doubling ratios between 6.7 and 12.3 for the pairs with $n \geq 200$). At $n = 750$, C took 0.41 s, Java 0.68–0.72 s and CPython 36.9 s; CPython did not finish $n = 1000$ within 60 s, whereas C and Java did in about 1.1 s and 1.7 s. Java's cold-start cost is visible below roughly 0.5 s of work and vanishes at the largest size. Memory grows quadratically in all three, with very different constants: about 4 bytes per element in C and Java plus a fixed runtime baseline (2 MiB for C, 42 MiB for the JVM), and much more per element in CPython. The slightly super-cubic growth of C and Python is consistent with cache effects but was not verified.

A Source code

A.1 C (`matrix.c`)

```
1 /* Basic triple-loop (i-j-k) matrix multiplication in C.
2  * Usage: ./matrix_c <n> [dump]
3  * Output (stdout): "<seconds> <checksum>"   (or the full result matrix with "
       dump")
```

```c
 * Inputs are deterministic so all languages multiply the SAME matrices:
 *   A[i][j] = (7*i + 3*j + 1) % 10 + 1
 *   B[i][j] = (5*i + 11*j + 2) % 10 + 1
 * checksum = sum_{i,j} (i+1)*(j+1)*C[i][j]   (64-bit integer)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static int** allocate_matrix(int n) {
    int** m = (int**)malloc((size_t)n * sizeof(int*));
    for (int i = 0; i < n; i++) m[i] = (int*)malloc((size_t)n * sizeof(int));
    return m;
}

static void free_matrix(int** m, int n) {
    for (int i = 0; i < n; i++) free(m[i]);
    free(m);
}

static void fill_matrices(int** a, int** b, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            a[i][j] = (7 * i + 3 * j + 1) % 10 + 1;
            b[i][j] = (5 * i + 11 * j + 2) % 10 + 1;
        }
}

static void multiply_matrices(int** a, int** b, int** c, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int sum = 0;
            for (int k = 0; k < n; k++) sum += a[i][k] * b[k][j];
            c[i][j] = sum;
        }
    }
}

int main(int argc, char* argv[]) {
    if (argc < 2) { fprintf(stderr, "Usage: %s <n> [dump]\n", argv[0]); return
        1; }
    int n = atoi(argv[1]);
    int dump = (argc > 2 && strcmp(argv[2], "dump") == 0);

    int** a = allocate_matrix(n);
    int** b = allocate_matrix(n);
    int** c = allocate_matrix(n);
    fill_matrices(a, b, n);

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    multiply_matrices(a, b, c, n);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    double secs = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;

    if (dump) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) printf(j ? " %d" : "%d", c[i][j]);
            printf("\n");
        }
    } else {
        long long checksum = 0;
```

```c
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) checksum += (long long)(i + 1) * (j + 1)
                * c[i][j];
        printf("%.6f %lld\n", secs, checksum);
    }
    free_matrix(a, n); free_matrix(b, n); free_matrix(c, n);
    return 0;
}
```

A.2 Java (main.java)

```java
/* Basic triple-loop (i-j-k) matrix multiplication in Java.
 * Usage: java main <n> [warmup] [dump]
 *   warmup = number of UNTIMED multiplications of the same size run before the timed one
 *            (0 = cold JVM: JIT compilation is included in the measurement).
 * Output (stdout): "<seconds> <checksum>"   (or the full result matrix with "dump")
 * Same deterministic inputs and checksum as matrix.c / matrix.py.
 */
public class main {
    public static void main(String[] args) {
        int n = Integer.parseInt(args[0]);
        int warmup = 0;
        boolean dump = false;
        for (int i = 1; i < args.length; i++) {
            if (args[i].equals("dump")) dump = true;
            else warmup = Integer.parseInt(args[i]);
        }

        int[][] a = new int[n][n];
        int[][] b = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                a[i][j] = (7 * i + 3 * j + 1) % 10 + 1;
                b[i][j] = (5 * i + 11 * j + 2) % 10 + 1;
            }

        int[][] c = null;
        for (int w = 0; w < warmup; w++) c = multiplyMatrices(a, b, n);   // untimed warm-up

        long t0 = System.nanoTime();
        c = multiplyMatrices(a, b, n);
        long t1 = System.nanoTime();
        double secs = (t1 - t0) / 1_000_000_000.0;

        StringBuilder sb = new StringBuilder();
        if (dump) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) { if (j > 0) sb.append(' '); sb.append(c[i][j]); }
                sb.append('\n');
            }
            System.out.print(sb);
        } else {
            long checksum = 0;
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++) checksum += (long) (i + 1) * (j + 1)
                    * c[i][j];
            System.out.println(String.format(java.util.Locale.US, "%.6f %d", secs, checksum));
```

```
        }
    }

    public static int[][] multiplyMatrices(int[][] a, int[][] b, int n) {
        int[][] result = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int sum = 0;
                for (int k = 0; k < n; k++) sum += a[i][k] * b[k][j];
                result[i][j] = sum;
            }
        }
        return result;
    }
}
```

A.3 Python (matrix.py)

```
"""Basic triple-loop (i-j-k) matrix multiplication in pure Python (no NumPy).

Usage: python3 matrix.py <n> [dump]
Output (stdout): "<seconds> <checksum>"   (or the full result matrix with "dump")
Same deterministic inputs and checksum as matrix.c / main.java.
"""
import sys
import time


def multiply_matrices(a, b, n):
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            s = 0
            for k in range(n):
                s += a[i][k] * b[k][j]
            result[i][j] = s
    return result


def main():
    n = int(sys.argv[1])
    dump = len(sys.argv) > 2 and sys.argv[2] == "dump"
    a = [[(7 * i + 3 * j + 1) % 10 + 1 for j in range(n)] for i in range(n)]
    b = [[(5 * i + 11 * j + 2) % 10 + 1 for j in range(n)] for i in range(n)]

    t0 = time.perf_counter()
    c = multiply_matrices(a, b, n)
    t1 = time.perf_counter()

    if dump:
        for row in c:
            print(" ".join(map(str, row)))
    else:
        checksum = sum((i + 1) * (j + 1) * c[i][j] for i in range(n) for j in range(n))
        print(f"{t1 - t0:.6f} {checksum}")


if __name__ == "__main__":
    main()
```

A.4 Peak-RSS wrapper (`memrun.c`)

```c
/* memrun: run a command and report its peak resident set size (ru_maxrss).
 * Replacement for `/usr/bin/time -f %M` (not installed everywhere).
 * Usage: ./memrun <cmd> [args...]
 * The child's stdout/stderr pass through; on exit memrun prints "MAXRSS_KB=<n>" to stderr.
 * Metric: peak RSS in KiB as reported by wait4()/getrusage (Linux), for the whole process
 * (JVM and interpreter included). memrun itself is tiny, so it does not inflate the value.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>

int main(int argc, char** argv) {
    if (argc < 2) { fprintf(stderr, "Usage: %s cmd [args...]\n", argv[0]); return 2; }
    pid_t pid = fork();
    if (pid == 0) { execvp(argv[1], argv + 1); perror("execvp"); _exit(127); }
    int status; struct rusage ru;
    if (wait4(pid, &status, 0, &ru) < 0) { perror("wait4"); return 2; }
    fprintf(stderr, "MAXRSS_KB=%ld\n", ru.ru_maxrss);
    return WIFEXITED(status) ? WEXITSTATUS(status) : 128 + WTERMSIG(status);
}
```